# What Should an Inspection Drone Fleet Learn? Tool Selection, Answer Fusion and Federated Reinforcement Learning under Time Budgets

Mahdi Khashan

[a][Affiliation to be added],

Abstract

Inspection drones can now run small vision models on board and send difficult images to a large model in the cloud. For every image, the drone must decide which model to ask, under a time limit and over a weak radio link. A popular idea is to let a vision-language model (VLM) make this decision as an agent and to train it with federated reinforcement learning across the fleet. This article tests that idea against simple alternatives on 13,729 power-line component images from the ElecVQA-Bench benchmark. We use four tools: a ResNet-50 classifier, the laya-vision decision model and SmolVLM-256M on the drone, and Qwen3-VL-8B in the cloud. We measure their latency, replay real LTE network traces, and count 10 points for a missed defect, 1 for a false alarm and 0.5 for sending an image to the operator. Three results stand out. First, a VLM agent becomes worse when it can call tools. Second, simple methods give the largest gains: combining the answers of all tools lowers the risk from 177 to 107 points per 1,000 images, and sending uncertain images to the operator lowers it to 92–94 points at 0.25 s per image. Third, deep reinforcement learning (PPO), trained centrally or with federated averaging over five drones, learns good tool choices but only matches well-tuned rules. It wins clearly in one of eight settings. The best methods answer an image in 0.18–0.25 s with 1.6–3.4 kB of upload, while a SmolVLM-based pipeline measured on a Jetson needs 1.4–6.4 s per video segment.

Keywords: drone inspection, edge AI, tool selection, federated learning, reinforcement learning, vision-language model

## 1. Introduction

Drones are changing how power lines, bridges and towers are inspected. They take many high-resolution images during a flight, and new drones can also analyse these images on board [1]. A drone can now use several AI models, which we call tools. A small image classifier is fast but makes mistakes. A vision-language model (VLM) on the drone is slower but can explain what it sees. A large VLM in the cloud is stronger, but the drone must send the image over a radio link that is often slow or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson board and need 1.4–6.4 s for each video segment [2]. Azimi et al. compare running such a model on the drone, in the cloud, or split between the two [3]. Both groups name an adaptive controller, which decides at run time which tool to use, as future work.

This article studies that controller. For every image, the drone must decide which tools to ask, when to stop and answer, and when to pass the image to a human operator. It must do this within a time budget and with a weak network. A fleet of drones could also learn this decision together with federated learning, without sharing images.

Two designs are possible. In the agentic design, a VLM is the agent: it looks at the image, asks other tools and reads their answers. In the routing design, a small policy only chooses the next tool, and the final answer is computed from the tools' own answers. We test both designs, and we compare classical methods (rules and logistic regression) with deep reinforcement learning (PPO [11]).

Our main contributions are:

- a test of a VLM decision model used as an agent, which shows that it becomes worse with tools;
- a simple fusion of tool answers with a threshold that needs no labels, which gives the largest single gain;
- a clear formulation of tool selection with time budgets, network delays and a limit on how many images go to the operator;
- a fair comparison of classical rules and deep reinforcement learning, central and federated, with latency and upload measured per image.

## 2. From Fixed Rules to Learned Tool Selection

Edge VLMs for inspection. EVLM [2] runs a LoRA-tuned SmolVLM-256M on a Jetson and reduces the upload of a 4 s video segment from 485 kB to 25 kB. The study in [3] measures SmolVLM on the drone, in the cloud, and split between them, under different bandwidths. INAR-VL [7] sends requests to the edge or the cloud based on

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

the input and the bandwidth. Farahani et al. [1] show that SmolVLM-256 needs about 2.8 s per image on a Jetson Orin, while a YOLO detector needs 0.14 s, and they conclude that a light detector should trigger the VLM only when needed.

Limitations of existing work: These systems use fixed rules to choose where a model runs. They do not decide per image which of several models to ask, and they do not learn this decision from the fleet's experience.

Routing and cascades. Federate the Router [18] trains routers for language models with federated averaging from offline scores. FedHLM [19] learns escalation thresholds in a federated way. BaRP [9] trains a router when only the chosen model's result is seen. Zhang et al. [8] study escalation along a chain of models when labels arrive only for some decisions.

Limitations of existing work: Most routers choose one model per request. They do not call several tools one after another within a time budget, and they rarely include a human operator as an option.

Learning methods. PPO [11] is a standard deep reinforcement learning method, and GAE [12] gives it more stable learning signals over long episodes. Federated averaging (FedAvg) [15] trains one model across clients by averaging their weights; it suffers when clients have different data [16]. Flower [14] provides federated simulation. The EM method of Saerens et al. [13] estimates how common defects are in new data without labels.

## 3. Data, Tools and Constraints

Data. We use the tower14 split of ElecVQA-Bench [20], built on InsPLAD power-line images [21]. It has 18 training towers (25,499 images), 10 validation towers (6,672) and 14 test towers (7,057), with 17 component types. Each image comes with a question about a defect. The validation towers contain 19.6% defects and the test towers 15.1%. We ran every tool once on every image and replay these saved answers.

Tools. Table 1 lists the four tools. The ResNet classifier always runs first. The other three are optional. We measured the time of each tool on a desktop GPU. Because we have no Jetson, we multiply onboard times by 9, our estimate of the speed gap to a Jetson Orin NX.

Network. The upload to the cloud follows real 4G/LTE traces [22]. We use a weak network: 10% of the trace bandwidth (about 3 Mbit/s) and six outages of 20 s per hour. A Qwen call sends a JPEG image of 33 kB (median), which takes about 85 ms. During an outage, Qwen cannot be used.

Risk. A missed defect costs 10 points, a false alarm 1 point, and sending the image to the operator (defer) 0.5 points. We report the risk per 1,000 images; lower is better.

Constraints. Table 2 lists the limits used in the experiments.

Table 1: The four tools. Time per image on the drone (onboard tools: desktop time multiplied by 9) or in the cloud (compute and round trip, without upload). Risk alone: risk per 1,000 test images when only this tool answers.

| Tool | Model | Runs on | Time | Risk alone |
|---|---|---|---|---|
| ResNet | ResNet-50 [23], fine-tuned | drone | 0.07 s | 177 |
| laya | laya-vision [6] (201M), fine-tuned | drone | 0.70 s | 261 |
| SmolVLM | SmolVLM-256M [4] with LoRA [24] | drone | 0.73 s | 185 |
| Qwen | Qwen3-VL-8B, no fine-tuning | cloud | 0.18 s | 626 |

Table 2: Constraints in the experiments.

| Constraint | Value |
|---|---|
| Time per image (single images) | 0.25, 0.75, 1.5 or 3 s |
| Mission segment | 40 images in a row from one tower |
| Time for a segment | 0.1, 0.15, 0.2 or 0.3 s per image, shared by all 40 images |
| Images sent to the operator | at most 5% or 2.5% per segment (2 or 1 images) |
| Classifier | runs on every image |
| Cloud | only when the link is up |

## 4. How the Drone Selects Tools

Combining answers (fusion). Each tool gives a probability that the image shows a defect. A logistic regression combines the probabilities of the tools called so far, with the question type and component type. We train it on the validation towers. Because the share of defects changes between towers, we correct the combined probability with the EM method [13], which estimates the defect share of the new towers without labels. The drone then reports a defect if the corrected probability is at least 1/11, the threshold that minimises the risk.

When to defer. Sending an image to the operator costs 0.5 points. Reporting an uncertain image costs more on average. Deferring is therefore the best choice when the defect probability lies between 0.05 and 0.5.

The decision problem. Box 1 gives the formal problem. In short: after the classifier, the drone can call laya, SmolVLM or Qwen, report, or defer. A tool can be called only if it fits in the remaining time. In a mission segment, the 40 images share one time budget and one limit on deferrals, and network outages last over several images.

Box 1: Problem formulation (used in all tool-selection experiments)

Risk of answer $a$ for an image with label $y$ (1 = defect):

$$r(a,y) = \begin{cases} 10 & a = \text{good},\ y = 1 \\ 1 & a = \text{defect},\ y = 0 \\ 0.5 & a = \text{defer} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Best answer for a defect probability $q$:

$$a^* = \begin{cases} \text{defer} & \min(10q,\ 1-q) > 0.5 \\ \text{defect} & q \geq 1/11 \\ \text{good} & \text{otherwise} \end{cases} \quad (2)$$

Fused probability for the set $C$ of called tools:

$$q_C = \mathrm{EM}\left(\sigma(w_C^\top x_C)\right) \quad (3)$$

Time of a cloud call at bandwidth $b$ (Mbit/s):

$$c_{\mathrm{Q}} = t_{\mathrm{Q}} + t_{\mathrm{rtt}} + \frac{8\,s}{10^6\,b} \quad (4)$$

Actions and reward when an image is answered:

$$\mathcal{A} = \{\text{report}, \text{call}_{\mathrm{L}}, \text{call}_{\mathrm{S}}, \text{call}_{\mathrm{Q}}, \text{defer}\}, \quad R = -r(a,y) - \lambda\frac{\tau}{B} \quad (5)$$

Constraints of a mission segment of $N = 40$ images:

$$\sum_{n=1}^{N} \tau_n \leq N\beta, \qquad \sum_{n=1}^{N} \mathbb{1}[a_n = \text{defer}] \leq \lfloor \delta N \rfloor \quad (6)$$

PPO objective with an optional rule term:

$$L(\theta) = \mathbb{E}[\ \min(\rho\hat{A}, \mathrm{clip}(\rho, 0.8, 1.2)\hat{A}) + c_H\mathcal{H}(\pi_\theta) + \alpha \log \pi_\theta(a^{\mathrm{rule}})] \quad (7)$$

| | |
|---|---|
| $q$, $q_C$ | defect probability; after fusing tools $C$ |
| $x_C$, $w_C$ | tool scores, question and component; weights |
| $t_{\mathrm{Q}}$, $t_{\mathrm{rtt}}$, $s$ | cloud compute time, round trip (50 ms), image size in bytes |
| $\tau$, $B$ | time used for the image; time budget |
| $\lambda$ | weight of time in the reward (0.01 or 0.03; 0 in segments) |
| $\beta$, $\delta$ | time per image; share of images that may be deferred |
| $\rho$, $\hat{A}$ | probability ratio new/old policy; advantage |
| $c_H$, $\alpha$ | entropy weight (0.01); weight of the rule term (0 or 0.5) |

What the policy sees. The learned policy receives the current fused probability and how close it is to the threshold, the classifier score, the scores of the tools already called, the time left, the bandwidth, whether the link is up, the time each tool would need, simple image statistics (size, blur, contrast, brightness), and the component type. In mission segments it also sees the images and deferrals left, how long the link has been in its current state, and how uncertain the last five images were.

Methods. Table 3 lists the methods we compare. All of them use the same fusion; they differ only in how they choose tools and deferrals.

Table 3: Methods compared. All use the same fused answer.

| Method | How it decides |
|---|---|
| Classical | |
| ResNet only | answers after the classifier |
| All tools | calls laya, SmolVLM and Qwen while they fit in the time |
| Uncertainty cascade | calls the next tool while the answer is close to the threshold |
| Cascade with defer | calls the next tool while uncertain; defers if no tool fits |
| Paced cascade | like the cascade, but gives each remaining image an equal share of the segment time; settings tuned on validation |
| Deep reinforcement learning (PPO) | |
| PPO from scratch | small neural network (about 7,000 weights) trained by trial and error |
| Warm-started PPO | first copies the paced cascade, then improves with PPO |
| Anchored PPO | warm-started PPO with an extra term that keeps it close to the rule |
| FedAvg PPO | the same, trained on five drones that share only weights |

Table 4: A VLM agent gets worse with tools. Report cost per 1,000 test images (defect risk plus errors on component and condition), 400 test images, 1 s budget, one seed.

| What PPO trains | Without tools | With tools |
|---|---|---|
| Scorer, central | 1,505 | 1,715 |
| Scorer, FedAvg | 1,505 | 1,628 |
| Decision head, central | 1,438 | 1,570 |
| Decision head, FedAvg | 1,438 | 1,725 |

## 5. Results

### 5.1. A VLM as the agent does not help

We first let laya-vision act as the agent. It looked at the image, chose to zoom, ask ResNet or ask Qwen, read the answers as text, and wrote the report. PPO trained its decision layer, centrally and with FedAvg over five drones in Flower. Table 4 shows the result: in every case the report with tools was worse than the report without tools. Separate decision layers for actions and answers did not help. A version of laya fine-tuned on our data also learned to call no tool, because every tool changed more correct answers into wrong ones than the reverse. Zoom also did not help the classifier: the defects it missed were in small, low-resolution images, so zooming added no detail.

### 5.2. Combining the tools' answers gives the largest gain

Table 6 shows that the tools are useful when their own answers are combined. Fusion lowers the risk from 177 (ResNet alone) to 114. The threshold needs care: the test towers have fewer defects than the validation towers, so a threshold tuned on validation gives too many false alarms. The EM correction estimates a defect share of 15.4% for

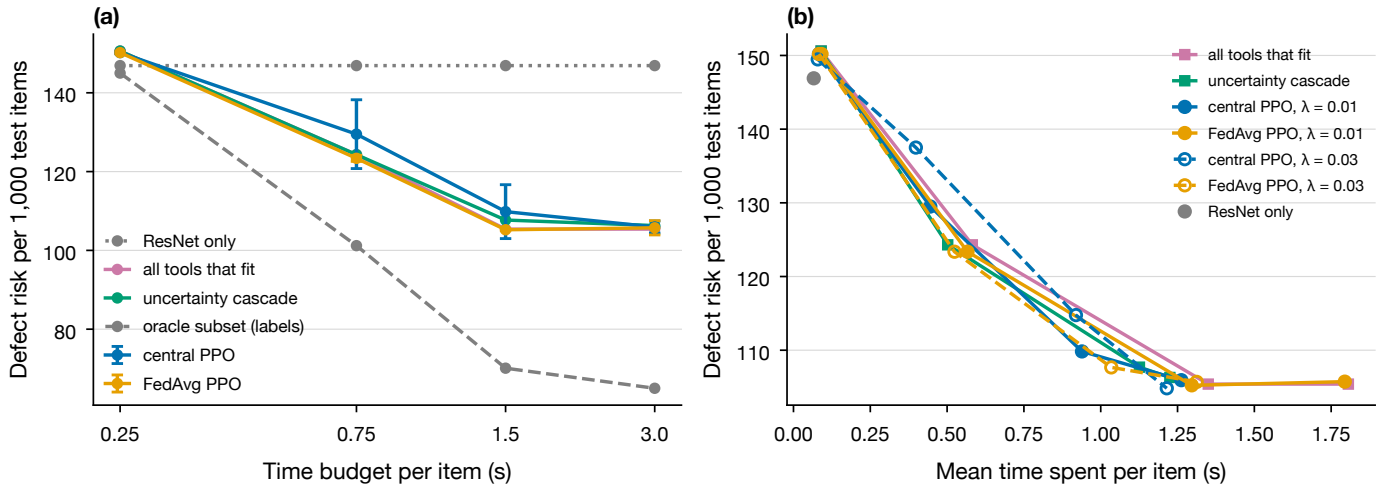



Figure 1: PPO learns to save time but not to lower the risk below a simple cascade. Without deferral. (a) Risk per 1,000 test images for each time budget. (b) Risk against the time spent per image; filled and open markers are two weights of time in the reward. PPO: mean of 3 seeds, bars show one standard deviation.

Table 5: Mission segments. Risk per 1,000 test images. Columns: share of images that may be deferred, and time per image shared by the 40 images of a segment. Learned methods: mean of 5 seeds (3 for PPO from scratch); the spread across seeds is 0.5–3.5 points. Bold: best in each column.

| | At most 5% deferred | | | | At most 2.5% deferred | | | |
|---|---|---|---|---|---|---|---|---|
| Method | 0.1 s | 0.15 s | 0.2 s | 0.3 s | 0.1 s | 0.15 s | 0.2 s | 0.3 s |
| Classical | | | | | | | | |
| ResNet only | 129.2 | 129.2 | 129.2 | 129.2 | 129.2 | 129.2 | 129.2 | 129.2 |
| Cascade with defer (greedy) | 117.8 | 109.5 | 108.8 | 96.9 | 121.0 | 114.1 | 109.6 | 99.1 |
| Paced cascade | 108.8 | 106.1 | 106.0 | 100.5 | 112.9 | 113.4 | 108.2 | 97.2 |
| Deep reinforcement learning (PPO) | | | | | | | | |
| From scratch, central | 122.8 | 123.0 | 120.7 | 114.8 | 122.1 | 120.3 | 121.4 | 115.4 |
| From scratch, FedAvg | 121.6 | 120.4 | 118.7 | 116.9 | 129.3 | 126.8 | 126.6 | 122.1 |
| Copy of the paced cascade | 112.3 | 107.6 | 104.0 | 97.9 | 114.5 | 115.8 | 109.8 | 99.8 |
| Warm-started, central | 112.5 | 109.5 | 107.0 | 99.7 | 117.1 | 114.7 | 112.1 | 102.9 |
| Warm-started, FedAvg | 117.1 | 111.8 | 107.5 | 101.0 | 118.0 | 113.5 | 112.3 | 103.9 |
| Anchored, central | 111.5 | 105.3 | 102.8 | 97.4 | 113.7 | 116.0 | 107.6 | 98.8 |
| Anchored, FedAvg | 112.7 | 106.5 | 102.9 | 98.3 | 114.9 | 115.2 | 109.3 | 99.2 |

Table 6: Combining tool answers, risk per 1,000 test images (7,057 images). 95% bootstrap intervals in brackets.

| Method | Risk |
|---|---|
| Best single tool (ResNet) | 177 (148–206) |
| Fusion, threshold tuned on validation | 116 (100–133) |
| Fusion, fixed threshold 1/11 | 114 (99–130) |
| Fusion, threshold with EM correction | 107 (91–124) |
| Fusion, best threshold (uses test labels) | 105 (89–124) |

the test towers (true value 15.1%) and lowers the risk to 107, almost as good as the best threshold chosen with test labels (105).

### 5.3. Selecting tools for single images

Next, the drone chooses tools for each image within its own time budget. Table 7 compares classical methods and PPO.

Deferring is the second large gain. Sending 7–11% of uncertain images to the operator lowers the risk by 11–15 points. The cascade with defer reaches 94 in only 0.25 s per image, because it defers instead of waiting for slow tools.

Table 7: Selecting tools for single images. Risk per 1,000 test images at two time budgets, and time and share of deferred images at 3 s. PPO: mean of 3 seeds (spread 1.6–4.7).

| | Risk | | At 3 s | |
|---|---|---|---|---|
| Method | 0.75 s | 3 s | Time | Deferred |
| Classical | | | | |
| ResNet only | 129 | 129 | 0.07 s | 0% |
| All tools | 114 | 108 | 1.81 s | 0% |
| Uncertainty cascade | 112 | 105 | 1.22 s | 0% |
| Cascade with defer | 99 | 94 | 0.25 s | 7% |
| All tools, then defer | 96 | 92 | 1.81 s | 11% |
| Deep reinforcement learning | | | | |
| PPO, small batches | 115 | 106 | 1.27 s | 0% |
| PPO, large batches | 106 | 95 | 0.37 s | 8% |

PPO matches the rules but does not beat them. With

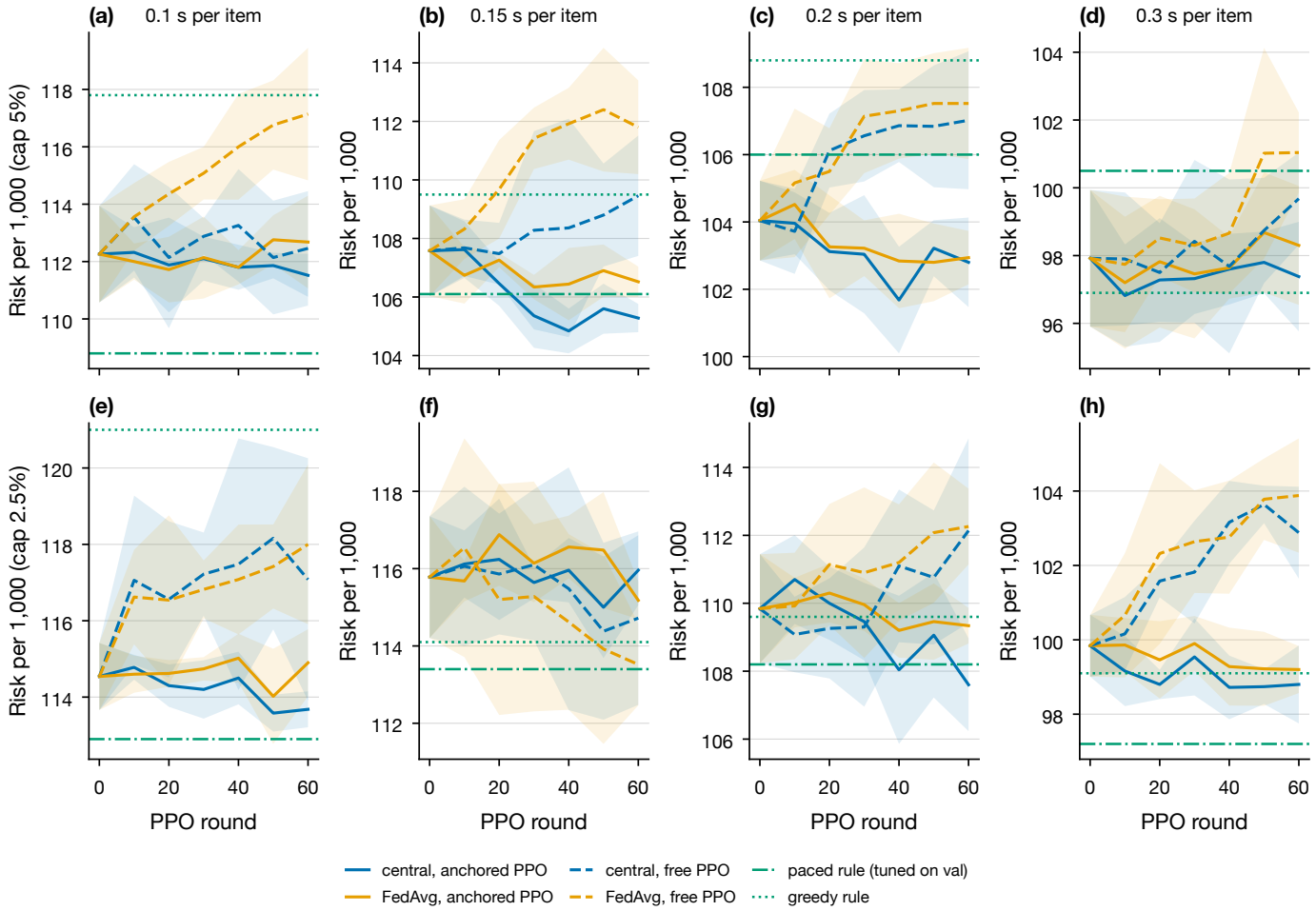



Figure 2: Warm-started PPO stays close to the rule it copied. Risk per 1,000 test images during PPO training; round 0 is the copy of the paced cascade. Top row: at most 5% deferred; bottom row: at most 2.5%. Columns: time per image. Solid lines: anchored PPO; dashed lines: PPO without the anchor. Blue: central; orange: FedAvg over five drones. Mean of 5 seeds, bands show one standard deviation. Green lines: paced cascade (dash-dot) and greedy cascade (dotted).

small batches (2,048 episodes per round), PPO never learns to defer. Deferring costs 0.5 on easy images, where reporting costs almost nothing, so PPO stops trying it too early. With four times larger batches, PPO learns to defer 8% of images and reaches 95, close to the rules. Without deferral, PPO reaches the risk of calling all tools in about 30% less time (Fig. 1), and FedAvg gives the same result as central training.

Why the oracle is not a fair target. An oracle that knows the label can pick the best set of tools for each image and reaches 65. But it is better than full fusion on only 127 of 7,057 images, and for 83% of these the fused probability is very close to the threshold. Only the label tells which side is right, so no method without labels can reach the oracle. Stronger fusion models (more features, gradient boosting) did not help either.

Drones that differ. We also gave each of five drones a different component group and a different network. The result did not change. FedAvg followed by a short local training was the most stable learned method and matched the cascade (108 against 106 at 3 s) with 15% less time, but no method lowered the risk. Rules tuned for each drone separately were worse (117), because each drone's validation and test towers differ.

### 5.4. Mission segments: shared time and operator limit

In a real mission, time and operator attention are shared. A good policy should save time on easy images and spend it on hard ones. Table 5 and Fig. 2 show the results for 40-image segments.

Planning matters. When time is short (0.1 s per image), the paced cascade is 8–9 points better than the greedy one, because it does not spend all the time on the first images.

PPO from scratch fails. It is 10–25 points worse than the best rule everywhere. It always uses all its time and all its deferrals, but on the wrong images. With the stricter operator limit, FedAvg learned never to defer.

Copying the rule works; PPO hardly improves it. When PPO first copies the paced cascade (98% of its actions), it starts close to the best rule. Further PPO training without

Table 8: Latency and upload. Prior work: measured on a Jetson Orin. This work: mean per test image, weak network.

| System | Time | Upload |
|---|---|---|
| Prior work | | |
| YOLO detector, per image [1] | 0.14 s | – |
| SmolVLM-256, per image [1] | 2.8 s | – |
| EVLM (SmolVLM), per 4 s segment [2] | 1.4–6.4 s | 25 kB |
| SmolVLM on the drone, per request [3] | 2.7–2.8 s | 0 |
| SmolVLM in the cloud, per request [3] | 0.8–3.9 s | image |
| This work | | |
| ResNet only | 0.07 s | 0 |
| All tools | 1.81 s | 34.5 kB |
| Cascade with defer | 0.25 s | 3.4 kB |
| PPO with defer (large batches) | 0.37 s | 9.4 kB |
| Paced cascade, segment (0.2 s) | 0.18 s | 1.6 kB |
| Anchored PPO, segment (0.2 s) | 0.18 s | 1.6 kB |

the anchor makes the policy worse in 14 of 16 settings. With the anchor, PPO changes the result by 2 points or less. It beats the best rule clearly in only one setting (5% deferred, 0.2 s per image: 102.8 against 106.0), and FedAvg gives the same result as central training. We chose the anchored version after seeing the test results, so even this one gain is optimistic.

### 5.5. Latency and upload

Table 8 compares the time and upload per image with published Jetson measurements. The units are not the same: an EVLM segment contains several frames and a text report, while our item is one image with one answer. Still, the best methods need 0.18–0.25 s and 1.6–3.4 kB per image, which is well within the 1.16 s between frames that EVLM assumes.

One number needs attention. Our onboard SmolVLM call takes 0.73 s with the factor of 9, but the Jetson measurements are 2.7–2.8 s, about 3.7 times longer. They include text generation and full frames, while our models only score answer options. If our onboard times are too low, only the largest budget would allow even one onboard VLM call per image, and selective calls and deferral would matter even more. A measurement on a Jetson is needed.

### 5.6. What a fleet should share

Table 9 compares what each federated option sends and what it gains. In earlier experiments on the same data, a small routing controller (29 KB) trained with FedAvg across five operator regions reached a risk of 115, equal to central training (115) and much better than a fixed cascade (156) or regions training alone (158), with about 7 MB of total upload. Under realistic feedback, where an accepted image reveals no label, only a 5% audit of accepted images kept it better than the cascade (142). Training the 201M-parameter laya-vision model itself with FedAvg cost 0.4–41 GB per run, and small updates doubled its calibration error.

Table 9: What a fleet can share, and the cost in upload.

| What is shared | Upload | Result |
|---|---|---|
| Defect share for the EM threshold | a few bytes | risk 114 → 107 |
| Tool-selection policy (PPO) | 28–30 KB per round | matches tuned rules |
| Routing controller | about 7 MB in total | 156 → 115 with full labels |
| Perception model (laya-vision) | 0.4–41 GB per run | better ranking, worse calibration |

## 6. Lessons and Open Challenges

Simple steps first. The largest gains came from classical methods: combining the tools' answers, correcting the threshold without labels, and deferring uncertain images to the operator. None of them needs reinforcement learning, and the EM correction needs only a few numbers from each drone.

Why rules are hard to beat. For a single image, the best next step depends almost only on how close the fused answer is to the threshold, and a cascade uses exactly this signal. Shared constraints add a planning problem, but the paced cascade already solves most of it. Learned policies are still useful: they adapt to the budget and the network without new tuning, and FedAvg trains them as well as central training with very small updates.

Agents need better tools, not only better learning. The VLM agent and the learned policies were limited by the same thing: on the hardest images, the tools disagree and nothing in the input shows which tool is right. Tools that add new information, for example higher-resolution views of small components, are more promising than a stronger learning method.

Limitations. The tool answers are saved and replayed; the fleet, the network and the onboard times are simulated, and the factor of 9 is an estimate. The risk weights are our choice. The tool-selection experiments use 3–5 seeds, the agent experiments one seed. All data come from one benchmark of power-line components.

## 7. Conclusion

This article asked what an inspection drone fleet should learn to use its AI tools well. A VLM that acts as an agent and reads the tools' answers became worse with tools. Simple methods gave the gains: combining the tools' answers with a label-free threshold lowered the risk from 177 to 107 per 1,000 images, and sending uncertain images to the operator lowered it to 92–94 in 0.25 s per image. Deep reinforcement learning, central or federated with updates of about 30 KB, learned to match well-tuned rules under time and operator limits, but beat them clearly in only one of eight settings. For tool selection, a fleet should first share the fusion and its threshold statistics and defer at the threshold; a learned policy is a cheap way to match a tuned rule, not yet a way to replace it.

## Data and code availability

The code, the cached tool answers and the result files are in the project repository, branches vision-fl and agentic-ppo-v2 (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [20, 21]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

## Declaration of competing interest

The author declares no competing interests.

## CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

## Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

## Acknowledgments

Experiments in this work were run and tracked with OpenResearch [25].

## References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).

[8] H. Zhang, S. Cha, H. B. Beytur, K. S. Chan, G. de Veciana, H. Vikalo, Online learning for multi-layer hierarchical inference under partial and policy-dependent feedback, arXiv:2603.04247 (2026).

[9] W. Wei, T. Yang, H. Chen, Y. Zhao, F. Dernoncourt, R. A. Rossi, H. Eldardiry, Learning to route LLMs from bandit feedback: One policy, many trade-offs, arXiv:2510.07429 (2025).

[10] A. Agarwal, H. B. McMahan, Z. Xu, An empirical evaluation of federated contextual bandit algorithms, arXiv:2303.10218 (2023).

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).

[12] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, in: International Conference on Learning Representations (ICLR), 2016.

[13] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[14] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[15] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[16] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[17] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[18] Askin, et al., Federate the router: Learning language model routers with sparse and decentralized evaluations, arXiv:2601.22318 (2026).

[19] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[20] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[21] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[22] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.

[23] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[24] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[25] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.